Why are Cloud Development Environments Spiking in Popularity, Now?

Tech companies are building their cloud development environments (CDEs) and dozens of vendors are launching their offerings. But why now?

GERGELY OROSZ

SEP 5, 2023 • PAID

Two months ago, we covered the quiet revolution unfolding at tech companies: cloud development environments (CDE.) We went through the ideas behind CDEs, their upsides and downsides, and case studies in the use of this tech at Uber, Slack and Pipedrive.

In this issue, we explore why CDEs are taking off now, and get insights from an engineer who has worked in the CDE space for 7 years, Mario Loriedo, senior principal software engineer at Red Hat, tech lead of open source project, Eclipse Che, and a Cloud Native Computing Foundation ambassador.

We cover:

1. Why are CDEs gaining popularity, now?
2. Virtual desktops vs CDEs
3. The evolution of the CDE space since 2016
4. The biggest challenges of the CDE space
5. The future of software development in local environments

1. Why are CDEs gaining popularity, now?

The concept of remote development environments stretches back to the dawn of computer programming. In the 1960s, expensive central computers were shared by

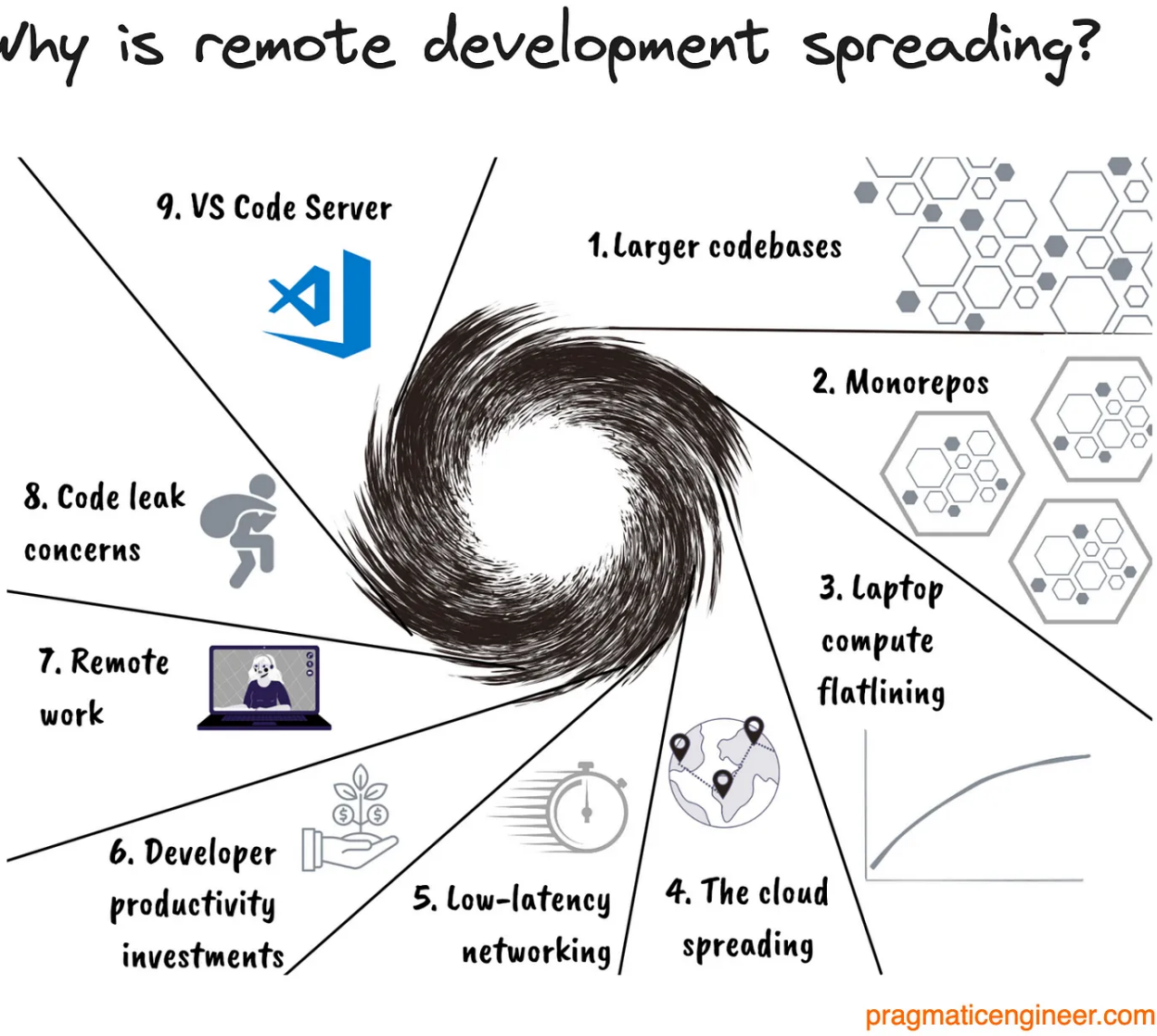
many users, sitting at terminals. Each user had a specific amount of time to use the processor in this setup called "timesharing."

Cloud development environments are similar to how programmers used a terminal to program a central computer, back then. A big difference is that in the case of old-school timesharing, the goal was to save on infrastructure costs, as the cost of CPU cycles was far higher than programming time. A single mainframe machine cost several times the annual salary of a programmer; the Atlas computer at Manchester University in the UK cost £50M in today's money, the equivalent of several hundred programmers' annual salaries. No wonder compute time was so valuable!

The input/output area of the Atlas computer (right) and the computer itself, occupying a large room with its circuit boards inside closets. Image source: The Atlas story

Today, it is compute that's much cheaper than software engineers' time. In the US, a software engineer working at a Big Tech company is compensated more than 100 times the cost of a high-end laptop, annually.

So why the sudden rise in use of remote environments? I observe several trends which make this shift understandable and sensible:



Reasons why remote development and cloud development environments are getting popular

Here are the reasons:

1. Larger codebases. Every day, there's more code at a tech company, not less. This means more repositories are needed, which are fast enough to build and work with, but which increase fragmentation. At many large companies, this inevitably leads to consolidating many smaller repositories into fewer, larger ones. Which leads to the next point...

2. Monorepos. These are becoming more popular at large tech companies for a few reasons. The biggest is consistency; with a monorepo, dependencies are clear and API breakages due to outdated dependencies are absent. Also, tasks like integration

testing are much easier. However, monorepos result in codebases growing large, so that even checking out the code or updating to the head can be time consuming.

3. Laptop compute power is plateauing. The available compute power of laptops and personal computers no longer increases as it did prior to 2010. Moore's Law says the number of transistors in an integrated circuit (IC) doubles around every two years. But the most recent 10 years suggest this is no longer the case.

4. The Cloud is spreading and offering more capabilities. Since around 2010, there's been a boom in Cloud computing. AWS launched in 2006, Azure in 2010, and GCP launched its first region in 2015. Since then, all Cloud providers have expanded their availability, and the price of Cloud compute has dropped due to competition and technological progress.

5. Low-latency networking is cheaper and more common. CDEs only make sense when latency is low. This requires high bandwidth internet connectivity for developers, and remote development environments to be located in close vicinity to servers. High-bandwidth networking is spreading rapidly, including fallback options with mobile 4G and 5G coverage. And with cloud providers launching new data centers, zones and regions, it's now easier than ever to have cloud environments near to software engineers' locations.

For example, Uber runs its Devpods out of 5 data centers this way, presumably operated by a provider like Google Cloud.

6. Developer productivity investments. Assuming a team of four software engineers costs $1M/year, would you spend $10K/year to improve the efficiency of that team by 10%? The answer should be a no-brainer 'yes,' because the business value generated by a team costing $1M/year should already exceed $1M/year. So if the team gets 10% more efficient, it should generate at least $100K/year extra value.

At companies with large codebases, CDEs are a pragmatic way to increase the productivity of software engineers.

7. Remote work. Covid-19 lockdowns and the rise of remote working have played a role in boosting CDEs. With remote work, engineers spend more time on video calls,

which utilizes laptop resources like CPU, memory, and more. Executing a build is much slower while on a call. Plus, a CPU and memory-intensive build can impact the quality of the video call, and make the local environment much less responsive.

LinkedIn gathered data showing how much slower video calls make common developer operations. It found video calls added a 2-4x increase in latency to a developer laptop.

8. Concern about code leaks. One downside of remote work – and hiring people whom you've not met in person – is that trust problems tend to arise more easily. When working from the office, it's easy to verify that the same person you hired is the one doing the work, remotely. With full-remote work, the risk is higher that someone other than the employee accesses the codebase.

CDEs can offer an additional layer of protection to reduce the risk of source code leaking outside of the network. At the very least, far more logging is in place, and it can be easier to detect when larger parts of the codebase are accessed and copied across the network.

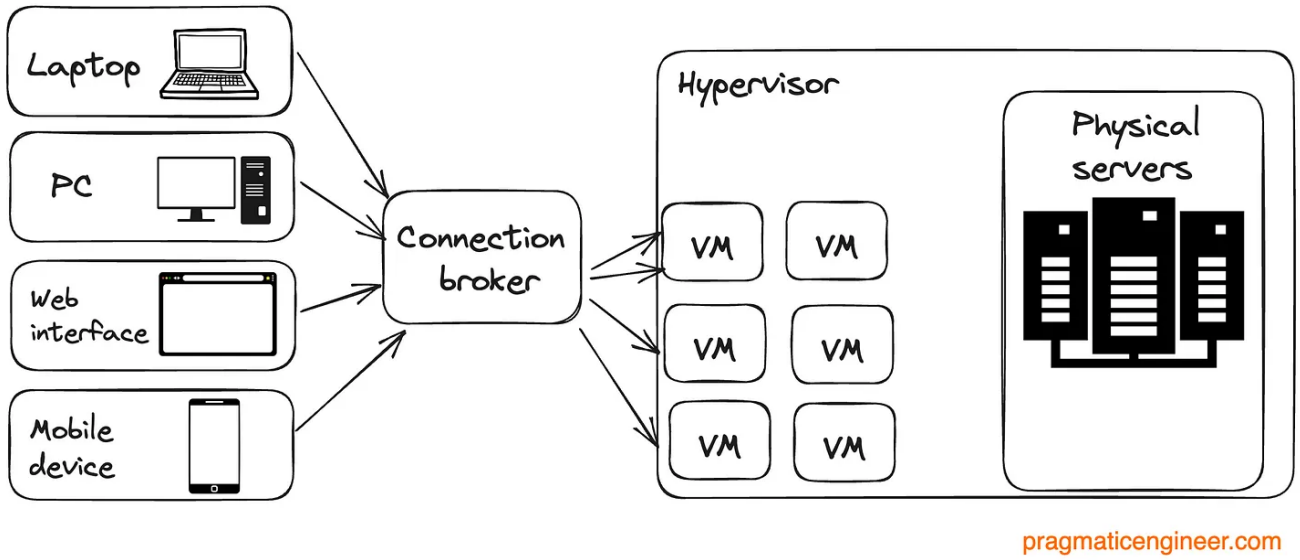
9. Open source VS Code Server. In 2021, Microsoft open sourced VS Code Server. This is the server code that powers VS Code remote development, and GitHub Codespaces. Thanks to this, the barrier to entry was dramatically reduced for *anyone* wanting to build their own cloud development environment which supports VS Code. This applies to startups offering CDE solutions, as well as companies building such a solution in house.

None of these trends alone can explain the spike in popularity of cloud development environments, but their combined effect in recent years does provide a plausible explanation. It's also possible to posit that the "spark" which ignited this confluence of factors was the Covid-19 pandemic of 2020-21, and the accompanying rise of remote work.

2. Virtual Desktops vs CDEs

One interesting thing about cloud development environments is that there's been a very similar technology to it in existence for nearly 20 years: virtual desktop

infrastructure (VDI.) This is a technology that runs virtual machines on a corporate server, and employees access the machine remotely:



A virtual desktop infrastructure solution

Components of a VDI solution:

- **Clients**: machines employees use to connect to virtual machines. These could be laptops, a web interface, a mobile device, or other.
- **Connection broker**: the component responsible for authentication and authorization, ensuring a secure connection (typically via Single-Sign-On, shortened to SSO,) and managing sessions and desktop pools.
- **Hypervisor**: also referred to as a virtual machine monitor (VMM.) The hypervisor segments physical servers into virtual machines. The hypervisor shares the underlying machines' resources of CPU, memory and disk, and creates multiple virtual machines on one physical machine.

The concept of virtualizing desktops gained momentum in the mid-2000s, with VMWare and Citrix being two notable vendors in the space. Here's a brief overview of the history of VDI from the book, Mastering VMware Horizon 7 by Peter von Oven and Barry Coombs:

> "The concept of virtualizing Windows desktops has been around since as early as 2002, when VMware customers started virtualizing desktop workloads and hosting them on a VMware server and ESX servers in the data center. As there was no concept of a connection broker at that time, and neither was the phrase VDI really used, customers simply connected using the RDP protocol directly to a dedicated desktop virtual machine running Windows XP.
>
> It wasn't until 2005 that VMware first showed the idea of having the concept of a connection broker. By demonstrating a prototype at VMworld, VDI entered the limelight, raising the profile of the technology. It was also at the same event that companies such as Propero showed their version of a connection broker. Propero would later become the Horizon View connection server.
>
> In early 2006, VMware launched the VDI alliances program, with a number of technology vendors such as Citrix, HP, IBM, Sun, and Wyse Technology joining this program.
>
> By 2007, the prototype connection broker was introduced to customers to help with development before it was given to the VMware product organization to productize it and turn it into a real product. The released product was called Virtual Desktop Manager 1.0 (VDM). (...)
>
> In 2010 (...) this was also the year that VMware talked publicly about the biggest VDI reference case to date with Bank of Tokyo Mitsubishi, who deployed 50,000 virtual desktop machines."

So it was around 2006-2007 that VDIs really started taking off across the industry. Their biggest benefits are:

- **Security**. The biggest driver for most VDI solutions is that what's on the virtual desktop, stays there. For example, source code doesn't move onto a developer's laptop, and there are no unauthorized installations of frameworks or libraries.
- **IT support**. With VDIs, IT staff do not have to support a host of different configurations, as the company will support a select few virtual desktop configurations.

- **Overall hardware cost**. It can be possible to save on overall hardware costs by having low-end laptops or PCs connect to beefy, on-prem servers.

VDIs have downsides, too:

- **Performance.** Working through a virtual machine can feel sluggish, and network latency is especially noticeable with a lower bandwidth internet connection, or when far away from the server running the virtual machine.
- **Less flexibility**. It's much more time and effort to get custom tools installed on a virtual machine, and this is often discouraged by design.

VDIs have been popular in industries where the benefits heavily outweigh the downsides, such as finance and banking for which security is particularly important.

However, CDEs offer benefits which VDIs do not:

- **Improved developer experience.** Developers can customize their development environment much better, and install tools on their local machine with far more flexibility than with VDIs.
- **More room to optimize performance.** Latency is as important with VDIs as with CDEs. However, it can be easier to run VMs closer to developers' machines with CDEs by utilizing public cloud provider infrastructure.
- **Potentially lower cost, overall.** CDE containers tend to be much shorter lived than virtual desktop machines, and often have lower resource needs. It can be cheaper overall than running a VDI – but not always.

These benefits make CDEs a tempting alternative to VDIs. And CDE is a "new kid on the block" technology, which makes it exciting and there's also more space for innovation. Therefore, I can see CDEs being suggested by engineers at companies already using VDIs.

3. The evolution of the CDE space since 2016

For this part of the article, we're joined by Mario Loriedo, senior principal software engineer at Red Hat. Mario has lived and breathed this space for 7 years, and brings an

expert viewpoint. But first, a little personal context.

How did you get into the developer productivity space?

'I used to work as a Java consultant in Paris around 2010. Back then, it could take literally days – sometimes weeks! – for new engineers to be able to push some code. I was looking at ways to accelerate developers' flow.

'The book "Continuous Delivery" by Jez Humble and David Farley, was the reference we passed around at the time. However, this book focused on the CI, rather than the inner loop of development.

'Vagrant by HashiCorp is a way to create and configure portable development environments. This was a solution to unify the inner loop of development, but it never got as popular as it deserved among devs. Then Docker changed things in 2014. It was not about CDE *yet*. While Docker was running on developers' laptops and not a remote environment, it allowed us to define "development environment as code". I sensed an increasing interest from customers, so I pivoted from Java to Docker consultancy.

'I created a couple of Docker plugins for IDEs – including for Eclipse and Sublime Text – and talked about it at a few conferences. This is how I got in touch with Red Hat. I joined the company in 2016 to work on a development platform based on OpenShift, Red Hat enterprise application platform powered by Kubernetes.'

What have you worked on at Red Hat?

'I joined Red Hat in the "developer tools" group. I started working on **Eclipse Che**, an open source project maintained by a company called Codenvy, which was running a container-based CDE service. In 2016, this was probably the first container-based CDE! Red Hat was interested in setting up a similar service running on top of Kubernetes. Everything aligned, as this was exactly the problem space I wanted to work in!

'In 2017, Red Hat launched a Developer platform called OpenShift.io to help engineers develop and test applications on Kubernetes. The platform included:

- a cloud IDE (Eclipse Che)

- a CI/CD pipeline (Jenkins)
- an issue tracker

'My team's goal was to adapt and maintain Eclipse Che so it could run on that platform. When we started Che, it could only run on Docker, so we had to add Kubernetes support. It was in this year that Red Hat acquired Codenvy, so our team grew significantly in size.

'In 2019, Red Hat Developer Sandbox replaced OpenShift.io. The cloud IDE, based on Eclipse Che, wasn't affected though and is still available at Workspaces.OpenShift.com.

'That year, the focus of my team shifted from running Eclipse Che as a service, to building a Red Hat product based on Eclipse Che. That included supporting on-prem installations which is a challenge because customer networks, their security policies and developer services, can vary a lot. Today we are about to release version 3.8 of OpenShift Dev Spaces, a mature product that runs well on restricted networks, supports Federal Information Processing Standards (FIPS) and allows developers to specify their cloud development environment using a *.devfile.yaml* in their git repo (Devfile is a Cloud Native Computing Foundation project.)'

How has the CDE space evolved since 2016?

'Quite a bit! Here's an overview of what I observed.

'In 2016, remotely-run IDEs were in their infancy. In 2016, it was startups like Codenvy, Cloud9 and others, which all developed their own IDEs that ran remotely. These were fully-featured IDEs with features like code completion, refactoring, code navigation.

'One development that helped these startups progress was the advent of the Language Server Protocol (LSP), which is an open, JSON-RPC-based protocol for use between IDEs and servers that provides "language intelligence tools" for things like code completion or syntax highlighting. In the early 2020s, LSP is the norm for language intelligence tools providers.

‘A drawback of these remote IDEs was that they all had stability and latency problems. Making a good cloud IDE takes time and requires a lot of experience. In my view, the only company that succeeded was Microsoft with VS Code, in around 2020. In 2016, the “performant” IDEs all ran locally, such as Eclipse, JetBrains, Sublime Text, Atom, or Visual Studio.

‘Since 2020 there’s been a step up in remote IDE performance and a hybrid approach. Since the start of the decade things have changed greatly. The two biggest players in the IDE industry (Microsoft and JetBrains) have released and open sourced versions of their IDE that run in a browser tab. The quality of these products is impressive.

‘To satisfy the needs of the most demanding developers – who won’t develop in a browser tab – these companies offer a hybrid approach. This means the IDE server runs remotely, and the IDE client runs on the developer’s laptop.

‘It is this hybrid change that has helped create a new breed of CDE startups. The business of such startups is about provisioning and configuring dev environments that have IDEs developed by Microsoft and JetBrains.

‘It’s safe to say that no major player develops its own IDE anymore. Those which don’t use Microsoft’s IDE (VS Code) for strategic reasons, almost always turn to an open source alternative such as Eclipse Theia.

‘Today, more major players are investing in remote development tools. Parallel to Microsoft and Jetbrains, major players like Gitlab, Amazon (AWS), Apple, Google and Docker, have invested in remote development tools.

‘My view is that in the next few years, we may have millions of developers running their tools in the Cloud. Cloud providers want developers to run tools in *their* cloud, which is why making investments like Amazon has, makes sense.

‘As an interesting observation, the most popular CDE providers such as GitHub Codespaces and Gitpod have chosen not to support on-premises. The reasons for this choice, in my opinion, are:

1. Running dev tools on-prem with strict security policies and custom processes is hard.
2. Customers are moving towards the Cloud anyway, so supporting on-prem isn't worthwhile.

'CDEs could replace virtual desktop infrastructure (VDI) solutions. Financial institutions have strict policies for developers and contractors, and used virtualized remote desktops for years. Virtual remote desktops allow the locking-down of development environments. This means source code doesn't transit on developers' laptops and developers tools, and that frameworks and runtimes are limited to the company-approved parameters.

'Compared to VDIs, CDEs provide a much better developer experience and have lower CPU/Memory/Disk costs.

'Conferences dedicated to CDEs are spreading. In the last couple of years, new conferences dedicated to CDEs included:

- Cloud Native DevX Day in 2021. A co-located event of KubeCon CloudNativeCon North America.
- DevX Conf in 2022
- CDE Universe in 2023

4. The biggest challenges of the CDE space

What are the biggest challenges you see for adoption of cloud development environments?

'CDEs are still very expensive. Some companies expect to reduce their costs when developers migrate to cloud development environments. But this is not what usually happens!

Also, developers won't give up their powerful laptop. And also, a CDE requires as many resources as a laptop does. Of course, resources in the Cloud are shared and can be optimized with container schedulers such as Kubernetes. Still, in practice it means that

even if the developers and ops are willing to use CDE, the budget for a CDE will be considered high. A big budget needs approval from the higher levels, which tends to slow down adoption.

'CDE is still immature. Today, a CDE is a single-tenant desktop IDE adapted to run the Cloud. There are still no multi-tenant, Cloud-native IDEs.

'For example, every time a developer starts a CDE, an instance of an IDE is created in a new container. To analogize this inefficiency, it's like opening Gmail in your browser tab, and an instance of the Eudora email client starting on a Google server. How well can this approach scale?

'We're used to multi-tenant applications in the Cloud. We also take for granted applications that can scale horizontally though replicas and load balancers. But when it comes to CDE infrastructure, these are not as advanced in resource allocation and up-and-down scaling.

'There is no standard file format to specify development environments – at least not yet. This is an important point worth noting. Dockerfiles and Compose are universally adopted file formats to specify *container* content and runtime. However, this format is not expressive enough to define a cloud development environment.

'There are efforts to standardize the specification, though:

- Dev Container Specification by Microsoft
- Devfile by the Cloud Native Computing Foundation

'Is Kubernetes the right platform to run cloud development environments? This is the other major concern about CDEs. As of today, Kuberntes is the most common platform for running CDEs. But is Kubernetes adapted to run development environments?

'Kubernetes is a platform designed to run unprivileged, containerized and scalable payloads. Is this platform *really* suited to run development environments that are notoriously single-user, monolithic applications? Are the benefits of lightweight,

ephemeral and scalable development environments worth the cost of altering the nature of Kubernetes to host developer tools?

'The answer to this question is still unresolved and controversial.'

5. The future of software development in *local* environments

This is Gergely again.

Thank you Mario for your expert insights. Readers can connect with Mario on LinkedIn.

It's clear the future of software development at large tech companies includes Cloud-based environments. At places like Slack, Uber and Shopify, the majority of software engineers are *already* building software using CDEs. But will the rest of the industry follow? Here's my thoughts.

CDEs have an overhead which doesn't make much sense for small teams. Even if we assume the infrastructure cost of running a Cloud-based environment is zero, it has sizable setup and maintenance costs. For small teams, this additional cost is unlikely to be justifiable. Having a CI/CD pipeline setup should work well enough when local build times are not a problem.

On the other hand, as a team and company grow, the setup and maintenance costs could be offset by faster iterations, easier onboarding, and switching between environments.

Larger tech teams will naturally gravitate towards building a CDE over buying one – for now. At companies with hundreds of software engineers working on the same codebase, the biggest developer pain points tend to be:

- Onboarding to the codebase as a new joiner
- How difficult integration tests are to do
- Too much time spent waiting on builds/tests
- Submitting a code change to a part of the codebase which another team owns

- Dependency updates or code changes that unexpectedly break other teams and are only discovered later

Monorepos offer some solutions to these issues, and CDEs offer solutions to others. It's unsurprising that large companies are able to make the business case for hiring entire engineering teams to solve these problems.

I do expect to see more vendors attempt to step up and solve for exactly this use case of large tech companies running their CDEs, while owning their own infrastructure as well. Similarly to how Buildkite filled a gap for mobile CI/CD solutions, tech companies stay in control of their infrastructure, and Buildkite builds and maintains the software to manage mobile build pipelines.

Medium-sized tech teams will be stuck on the fence. For large teams/companies, CDEs make a lot of sense. For small ones, not so much. But what about companies in between?

There's no one answer as to whether or not medium-sized tech teams should go with CDEs. In the previous article on CDEs, we covered a company with around 200 engineers called Pipedrive, which was happy with the transition to build its own CDE. However, companies that don't face pain points which CDEs can solve, like slow build times and local environment issues, might not lose much by sticking to development on local machines.

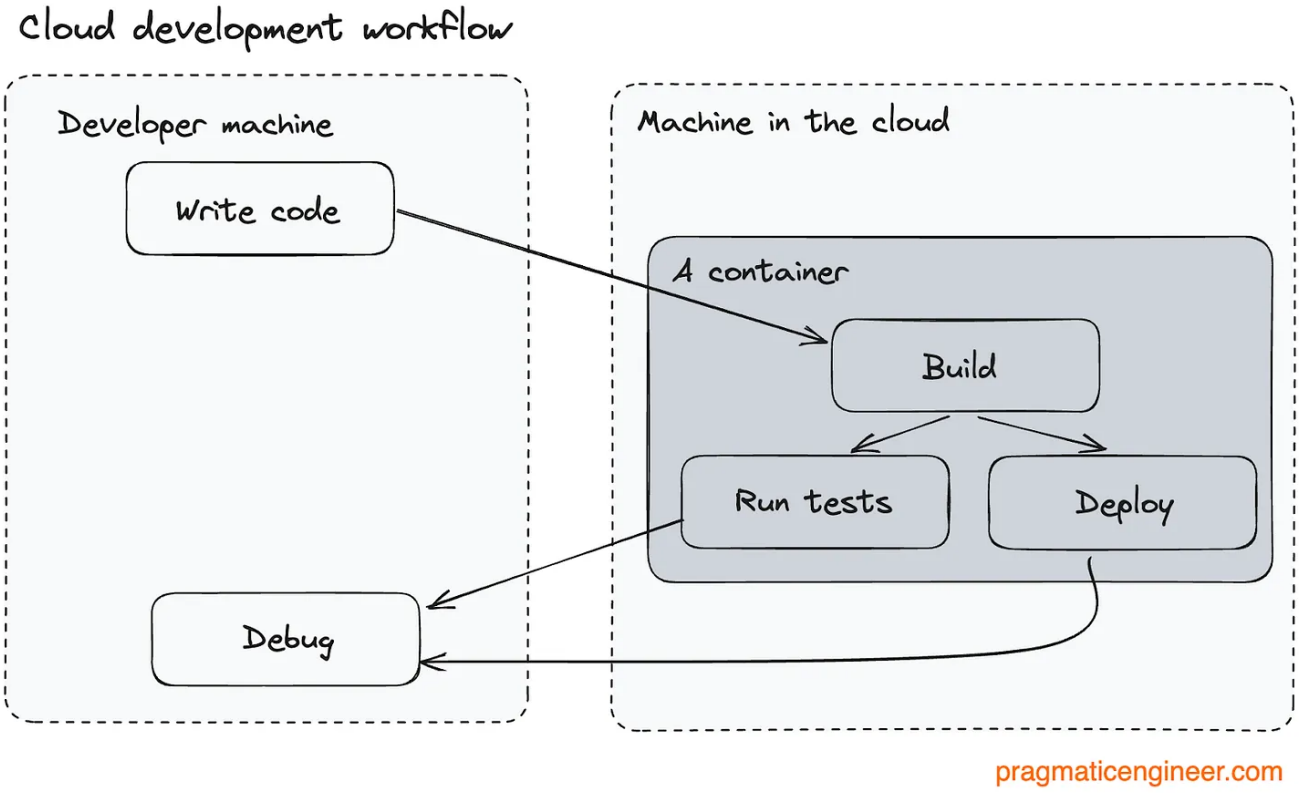
CDE solutions are still early-stage. One of the best-known CDE vendors, Gitpod, was founded in 2020, while GitHub Codespaces launched only in 2021.

It's easy enough to see how young the market is from how many companies are building in-house CDE solutions. They do this because they have more confidence in their own infrastructure teams than in vendors which are new to the market.

Also, there is still plenty to be desired about CDEs from a developer experience point of view. For example, many CDE vendors only work when connected to the internet, meaning that if a CDE has an outage, then all development across the company grinds to halt.

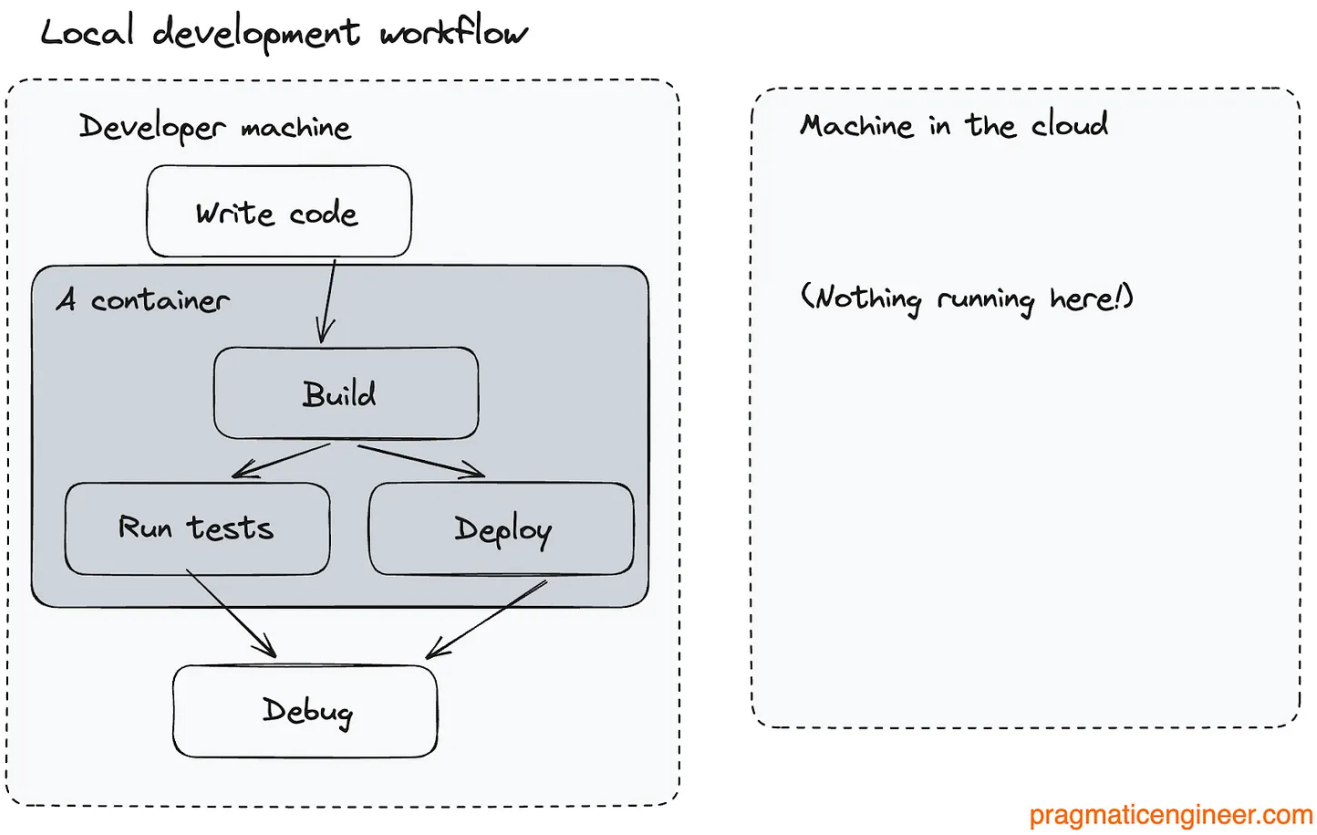
Easy-to-set-up local development environments will keep evolving. What if the future of Cloud development isn't *just* running builds in the Cloud? What if there was the option to remove the "Cloud" and run it locally?

A developer or team could do development on a remote machine like this:



The current cloud development workflow

And switch back to local development any time, like this:



What if there was an easy way to “switch” between developing on the Cloud and developing locally, while keeping the benefits of CDE locally, except for increased performance?

Ideally, switching between the Cloud and local development would be seamless. If such a solution existed and resulted in fast-enough local development speed, it would be a no-brainer for smaller teams to adopt.

In cases where boosted performance isn’t needed, the benefits of Cloud development like consistency and easy onboarding, could be enjoyed even without spending on Cloud infrastructure. But right now, the reality is that many CDE vendors carry the risk of Cloud lock-in, which is why smaller and medium-sized organizations should be wary of rushing ahead with CDEs.

Takeaways

The rise of Cloud-based development environments and remote development are very recent trends, and adoption seems to be accelerating at many large tech companies, and in some medium-sized ones, too.

In a follow-up article, we'll explore the fast-evolving vendor landscape, which has seen an explosion in CDE providers, including open source alternatives, and insights from vendors themselves.

15 Likes · 2 Restacks

1 Comment

Write a comment...

Dan Miller 3 hrs ago ♥ Liked by Gergely Orosz

I work at a small startup and we invested early on (less than a year in) in a custom CDE just so we could give all developers a Linux environment. Developing on a Mac but targeting Linux has worked fine historically, but the growing use of Docker/Kubernetes in the local development flow has made macOS painful due to the fact that you need to run a Linux virtual machine all the time.

We're much happier. It's easy to distribute updates to the CDE, and our developers can work on battery power for most of a workday instead of just a couple hours.

LIKE (1) REPLY •••